Windows setup: company Ethernet internet and USB LTE private network

Reference date: 10 November 2026. Applies to Windows 10/11 with the built-in NetTCPIP PowerShell module.

Purpose

Use company Ethernet for internet access and company resources while accessing a private SIM network through a USB LTE router. Windows selects the connection according to the destination IP address.

This is a reference procedure with example addresses. It has not been applied to this PC. Replace the examples with your actual adapter, gateway, and private network prefix before running configuration commands.

```
Company network / Internet
          |
     Ethernet adapter
          |
       Windows PC
          |
     USB network adapter
          |
       LTE router
          |
     Private SIM / APN
          |
 Private device 10.50.20.30
```

The PC does not need network bridging, Internet Connection Sharing, or IP forwarding. Keep these disabled unless a separate design specifically requires them. Follow company policy for simultaneous network connections.

1. Requirements and information to collect

- The router must support USB networking/tethering. A USB port used only for power, storage, or a serial console is insufficient.
- Windows must recognise the router as a network adapter. Install the manufacturer's driver if required.
- The SIM must be provisioned for the private APN, and the router must use the correct APN and authentication.
- The private network must permit access to the target device from this SIM. Private SIMs are not necessarily allowed to communicate with each other.
- You need local administrator rights to change Windows routes and interface settings.
- Obtain the exact destination subnet and mask from the private network administrator. An address beginning with `10` does not establish its subnet.

Record these values:

Item	Worked example	Your value
Company adapter alias	Ethernet	
Company IPv4 address	172.20.10.45	
Company gateway	172.20.10.1	
USB adapter alias	Ethernet 2	
USB adapter index	17	
PC address on USB network	192.168.8.100/24	
Router USB-side gateway	192.168.8.1	
Remote private network	10.50.0.0/16	
Remote device	10.50.20.30	
Device TCP service	HTTPS, port 443	
Private DNS domain/server, if needed	Obtain from administrator	

The USB gateway is the router address reachable directly from the PC. It is **not** normally the router's cellular WAN address or the remote device address. Leave USB addressing on DHCP when the router provides it.

2. Understand the route scope

Prefix	Addresses selected	Typical use
10.50.20.30/32	One device	Narrowest choice for one target
10.50.20.0/24	10.50.20.0–10.50.20.255	One subnet
10.50.0.0/16	10.50.0.0–10.50.255.255	Several related subnets
10.0.0.0/8	All 10.x.x.x addresses	Only when explicitly required

Windows prefers the most specific matching route. Metrics help choose between otherwise equivalent routes; lowering a metric does not override a more specific destination prefix.

Avoid adding `10.0.0.0/8` just because the target starts with `10`. Company networks and VPNs often use that range. If both networks contain the same destination IP, a route cannot distinguish the intended service by address alone. Coordinate address changes, translated addresses, or an isolated access environment with IT.

Also check the USB subnet itself: if the company network uses `192.168.8.0/24`, change the router's USB/LAN subnet to a non-overlapping range before proceeding.

3. Inspect and record the current configuration

Connect company Ethernet and confirm normal internet access first. Connect the USB router and enable USB tethering if its settings require it.

Open **Start → Windows PowerShell → Run as administrator**. Run:

```
Get-NetAdapter | Format-Table Name, InterfaceDescription, ifIndex, Status
Get-NetIPConfiguration
Get-NetIPInterface | Format-Table InterfaceAlias, InterfaceIndex, AddressFamily, Dhcp, AutomaticMetric,
InterfaceMetric, IgnoreDefaultRoutes
Get-NetRoute -AddressFamily IPv4 | Sort-Object DestinationPrefix | Format-Table DestinationPrefix,
InterfaceAlias, NextHop, RouteMetric
Get-DnsClientServerAddress
```

Identify the USB adapter by its description and the address assigned by the router. Adapter names vary; a USB device may appear as `Ethernet 2`, `USB Ethernet`, or a manufacturer-specific name. If uncertain, compare the adapter list before and after connecting the router.

Save the baseline to a new folder on your Desktop:

```
$snapshotPath = Join-Path ([Environment]::GetFolderPath('Desktop')) ('Network-before-LTE-' + (Get-Date
-Format 'yyyyMMdd-HHmmss'))
New-Item -ItemType Directory -Path $snapshotPath | Out-Null
Get-NetIPConfiguration | Format-List * | Out-File (Join-Path $snapshotPath 'ip-configuration.txt')
Get-NetIPInterface | Export-Clixml (Join-Path $snapshotPath 'ip-interfaces.xml')
Get-NetRoute | Export-Clixml (Join-Path $snapshotPath 'routes.xml')
Get-DnsClientServerAddress | Export-Clixml (Join-Path $snapshotPath 'dns.xml')
```

These files are records for comparison, not an automatic restore script. In particular, record the original USB `IgnoreDefaultRoutes` settings for IPv4 and IPv6.

4. Define the actual connection values

The remaining commands use these variables. Edit them first:

```
$usbAlias = 'Ethernet 2'
$usbIndex = (Get-NetAdapter -Name $usbAlias -ErrorAction Stop).ifIndex
$usbGateway = '192.168.8.1'
$privatePrefix = '10.50.0.0/16'
$privateDevice = '10.50.20.30'
$servicePort = 443

Get-NetIPConfiguration -InterfaceIndex $usbIndex
```

Confirm the displayed adapter and gateway match the USB router. Do not continue if they identify company Ethernet. Variables must be entered again if you open a new PowerShell session.

5. Prevent USB from supplying the default route

Set the USB interface to ignore dynamically supplied IPv4 default routes:

```
Set-NetIPInterface -InterfaceIndex $usbIndex -AddressFamily IPv4 -IgnoreDefaultRoutes Enabled
```

If the USB adapter has IPv6 enabled, apply the same setting to IPv6:

```
Get-NetIPInterface -InterfaceIndex $usbIndex -AddressFamily IPv6
Set-NetIPInterface -InterfaceIndex $usbIndex -AddressFamily IPv6 -IgnoreDefaultRoutes Enabled
```

Skip the IPv6 change if there is no IPv6 interface. Disabling IPv6 globally is unnecessary.

This setting controls dynamic default-route addition. Check for an already existing USB default route; do not assume the setting removed it:

```
Get-NetRoute -InterfaceIndex $usbIndex -AddressFamily IPv4 -DestinationPrefix '0.0.0.0/0' -ErrorAction
SilentlyContinue
Get-NetRoute -InterfaceIndex $usbIndex -AddressFamily IPv6 -DestinationPrefix '::/0' -ErrorAction
SilentlyContinue
```

If an existing USB default route remains, remove only that adapter's active default route. The commands ask for confirmation:

```
Get-NetRoute -InterfaceIndex $usbIndex -AddressFamily IPv4 -DestinationPrefix '0.0.0.0/0' -PolicyStore
ActiveStore -ErrorAction SilentlyContinue |
    Remove-NetRoute -Confirm

Get-NetRoute -InterfaceIndex $usbIndex -AddressFamily IPv6 -DestinationPrefix '::/0' -PolicyStore
ActiveStore -ErrorAction SilentlyContinue |
    Remove-NetRoute -Confirm
```

Check whether a manually saved USB default route also exists:

```
Get-NetRoute -InterfaceIndex $usbIndex -PolicyStore PersistentStore |
    Where-Object { $_.DestinationPrefix -in @('0.0.0.0/0', '::/0') }
```

If one is listed, investigate why it was configured before removing it. A saved default route may reappear after restart. Do not delete company or VPN routes.

Confirm company Ethernet still supplies the intended default path:

```
Get-NetRoute -AddressFamily IPv4 -DestinationPrefix '0.0.0.0/0'
Get-NetRoute -AddressFamily IPv6 -DestinationPrefix '::/0' -ErrorAction SilentlyContinue
```

IPv6 default routes may be absent on an IPv4-only company network. If no usable company default route exists, restore company connectivity before proceeding. This design intentionally provides no LTE internet fallback.

6. Test a temporary private route

Create a route for the current Windows session only:

```
New-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -NextHop $usbGateway -
PolicyStore ActiveStore
```

If Windows reports that the route already exists, inspect it before making another change:

```
Get-NetRoute -DestinationPrefix $privatePrefix
```

If the private device is already on the PC's directly connected USB subnet, Windows normally already has an on-link route; an additional gateway route is unnecessary. The worked example assumes the device is behind the router across LTE.

Check route selection

```
Test-NetConnection -ComputerName $privateDevice -DiagnoseRouting -InformationLevel Detailed
```

The selected outgoing interface should be the USB adapter, and the selected source address should be its address, such as `192.168.8.100`. Compare the interface index with `$usbIndex`.

Check the actual service

```
Test-NetConnection -ComputerName $privateDevice -Port $servicePort -InformationLevel Detailed
```

Expected example:

```
RemoteAddress    : 10.50.20.30
RemotePort       : 443
InterfaceAlias   : Ethernet 2
SourceAddress    : 192.168.8.100
TcpTestSucceeded : True
```

Use a TCP port the device actually provides. For example, SSH commonly uses 22 and Modbus TCP commonly uses 502; use the device's configured value. TCP success confirms connection establishment, not application authentication or correct application data. This command does not test UDP services.

Check company internet and resources

```
Test-NetConnection -ComputerName www.microsoft.com -Port 443 -InformationLevel Detailed
```

Confirm the selected interface is company Ethernet. Also open a normal website in your browser and test a company resource you use. Companies requiring a web proxy may block this direct TCP test even when browser internet access works.

7. Make the verified route persistent

After the temporary route works, replace it with a saved route:

```
Remove-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -NextHop $usbGateway -
PolicyStore ActiveStore -Confirm
New-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -NextHop $usbGateway
```

Omitting `PolicyStore` on `New-NetRoute` creates the route in both the active and persistent stores. Do not use `-PolicyStore ActiveStore` for the final saved route.

Check both stores:

```
Get-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -PolicyStore ActiveStore
Get-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -PolicyStore PersistentStore
```

Reconnect the USB router, then repeat the internet and private-service checks. Restart Windows when convenient and repeat them again. Recheck `IgnoreDefaultRoutes` and the default routes after reconnection and restart. USB driver software or company policy can reapply settings.

Adapter identity or addressing can change after driver reinstallation, changing USB modes, or replacing the router. Reinspect the adapter and gateway in that case; a saved route is not proof that it still points to the intended device.

8. DNS and private hostnames

Start with a numeric private IP to separate routing from name resolution. A working IP connection does not guarantee private names will resolve.

```
Get-DnsClientServerAddress
Resolve-DnsName 'device.example.internal'
```

The domain above is a placeholder. If the private network supplies a DNS server, first ensure its IP is reachable through a suitable USB route, then test it directly:

```
Resolve-DnsName 'device.example.internal' -Server '10.50.0.53'
```

Do not replace company DNS globally to fix one private name. Where private names are required, ask IT for a domain-specific DNS configuration, such as an NRPT rule, using the actual private suffix and DNS server. USB DHCP may also supply DNS settings; ignoring default routes does not suppress those DNS settings. Check DNS separately if company names stop resolving.

9. Troubleshooting

Symptom	Check and next action
No USB network adapter	Enable tethering; check cable data support, router USB mode, and driver in Device Manager.
USB address is 169.254.x.x	DHCP likely failed. Check router DHCP and USB networking rather than inventing a static address.

Symptom	Check and next action
Route uses company Ethernet	Confirm destination prefix, USB index, gateway, and more specific company/VPN routes.
USB route selected, service fails	Check APN registration, SIM access controls, router forwarding/firewall, remote device firewall, and service port.
Ping fails but TCP works	ICMP may be blocked; use the actual application service as the acceptance test.
Router is reachable but remote device is not	Router management reachability only proves the local USB path. Check LTE/private-network access separately.
Internet stops after USB connection	Inspect IPv4 and IPv6 defaults, DNS, VPN routes, and company endpoint policy.
Numeric private IP works but hostname fails	Configure the appropriate private DNS path and suffix with IT.
Settings fail after reconnect/restart	Recheck interface identity, saved route, default-route settings, router DHCP, and driver/policy overrides.
Adding route reports an existing object	Inspect active and persistent stores; do not repeatedly add the same route.
Only works without the company VPN	VPN routes or policy may override/block local connections; coordinate with IT.

The router must either translate the PC's USB source address appropriately or the private network must have a return route to that USB subnet. Windows routes alone cannot create the return path or bypass SIM isolation.

Useful diagnostic commands:

```
ipconfig /all
route print -4
route print -6
tracert -d 10.50.20.30
```

Traceroute can time out where routers block diagnostic traffic; timeouts alone do not prove the application path is broken. Do not disable Windows Firewall as a general troubleshooting step. Add only a specifically required rule if the application and policy require it.

10. Roll back this setup

Re-enter the variables from section 4 using the current adapter identity. Remove only the private route created by this procedure, from both stores:

```
Get-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -NextHop $usbGateway -
PolicyStore PersistentStore -ErrorAction SilentlyContinue |
    Remove-NetRoute -Confirm

Get-NetRoute -DestinationPrefix $privatePrefix -InterfaceIndex $usbIndex -NextHop $usbGateway -
PolicyStore ActiveStore -ErrorAction SilentlyContinue |
    Remove-NetRoute -Confirm
```

Restore the original USB default-route settings recorded in section 3. If they were originally `Disabled`, use:

```
Set-NetIPInterface -InterfaceIndex $usbIndex -AddressFamily IPv4 -IgnoreDefaultRoutes Disabled
# Run only if IPv6 was changed and its original setting was Disabled:
Set-NetIPInterface -InterfaceIndex $usbIndex -AddressFamily IPv6 -IgnoreDefaultRoutes Disabled
```

Reconnect the USB router so it can obtain its normal DHCP/router configuration. Re-enabling dynamic default routes does not itself recreate a manually configured default route removed earlier. Restore any such route only from the recorded original configuration.

Finally verify company internet access, company resources, and the route table. Disconnecting USB immediately removes that physical access path, but does not remove saved route/interface configuration.

11. Acceptance checklist and maintenance record

- ☐ USB adapter, gateway, and private prefix confirmed from actual configuration.
- ☐ Company and USB networks checked for address overlap.
- ☐ Baseline and original default-route settings recorded.
- ☐ Internet and company resources work through Ethernet.
- ☐ Private service works through USB using its numeric IP.
- ☐ Private DNS tested separately if required.
- ☐ IPv4 and IPv6 default-route selection checked.
- ☐ Reconnection and Windows restart checks completed.
- ☐ Actual values and any additional changes recorded below.

Date	PC/router	Private prefix and gateway	Validation / changes

Microsoft references

- New-NetRoute: destination, next hop, and route persistence
- Set-NetIPInterface: IgnoreDefaultRoutes and interface settings
- Test-NetConnection: TCP tests and route diagnostics

Command behavior was checked against these references on the reference date. Network examples and the troubleshooting workflow are illustrative and must be adapted to the actual router, SIM service, and company network.